

Azure Functions – Overview (Instructor Explanation)

1. What is Azure Functions?

Azure Functions is a **serverless compute service** provided by Microsoft Azure that allows you to **run small pieces of code (functions)** in response to events **without managing servers or infrastructure**.

You focus on **writing business logic**, and Azure automatically handles:

- Server provisioning
- Scaling
- Patching
- Availability

This model is commonly referred to as **Function-as-a-Service (FaaS)**.

2. Key Concept: Serverless

Serverless does not mean “no servers.”

It means:

- You do **not manage** the servers
 - Azure manages infrastructure, OS, runtime, and scaling
 - You pay only for **execution time and resources used**
-

3. When Should You Use Azure Functions?

Azure Functions are ideal for:

- Event-driven processing
- Background jobs
- Lightweight APIs
- Automation tasks
- Integration workflows

Typical scenarios:

- Process a message from a queue
 - React to a file upload in Blob Storage
 - Run a scheduled job (cron)
 - Handle HTTP requests (REST APIs)
 - Process events from Event Grid or Service Bus
-

4. Core Building Blocks of Azure Functions

a. Function App

- A **Function App** is a container for one or more functions
- All functions inside a Function App:
 - Share the same runtime
 - Share configuration settings
 - Share scaling behavior

Think of it as similar to an **ASP.NET Core Web App**, but for functions.

b. Function

- A **function** is a single unit of execution
- Written to perform **one specific task**
- Can be triggered by different event sources

Example:

- One function → HTTP request
 - Another function → Timer trigger
-

c. Triggers

A **trigger** defines **what starts a function**.

Common triggers:

- HTTP Trigger
- Timer Trigger
- Blob Storage Trigger
- Queue Trigger
- Service Bus Trigger
- Event Grid Trigger
- Cosmos DB Trigger

Each function has **exactly one trigger**.

d. Bindings

Bindings provide a **declarative way** to connect your function to other services.

Types:

- **Input bindings** – read data
- **Output bindings** – write data

Example:

- Read a message from a queue (input binding)
- Save data to Blob Storage (output binding)

Bindings reduce boilerplate code and improve productivity.

5. Supported Languages

Azure Functions supports multiple languages:

- C# (.NET)
- JavaScript / TypeScript
- Python
- Java

- PowerShell

For your background in **ASP.NET Core**, **C# Azure Functions** will feel most natural.

6. Execution Models

a. In-Process (.NET)

- Runs inside the Azure Functions runtime
- Faster startup
- Tightly coupled with runtime
- Recommended for existing projects (but less future-proof)

b. Isolated Process (.NET)

- Runs in a separate process
 - Better control over dependencies
 - Cleaner startup configuration
 - Recommended for **.NET 7 / .NET 8**
-

7. Hosting & Scaling Plans

a. Consumption Plan

- Default serverless model
- Auto-scale
- Pay per execution
- Cold start may occur

b. Premium Plan

- Pre-warmed instances (reduced cold start)
- Better performance
- VNET support

c. App Service Plan

- Dedicated servers
 - No cold start
 - Similar to hosting a Web API
-

8. Pricing Model

You are charged based on:

- Number of executions
- Execution duration
- Memory consumption

Consumption Plan includes **free grants**, making it cost-effective for low to medium workloads.

9. Azure Functions vs ASP.NET Core Web API

Aspect	Azure Functions	ASP.NET Core Web API
Architecture	Event-driven	Request-driven
Server Management	Fully managed	You manage
Scaling	Automatic	Manual/Auto-scale
Use Case	Small tasks, automation	Full REST APIs
Cost Model	Pay-per-use	Fixed hosting cost

10. Monitoring & Logging

Azure Functions integrates with:

- **Application Insights**
- Azure Monitor
- Live metrics
- Distributed tracing

This provides:

- Request tracking
 - Failure analysis
 - Performance monitoring
-

11. Common Real-World Use Cases

- API endpoints for microservices
 - File processing pipelines
 - Background email/SMS processing
 - Scheduled data cleanup jobs
 - Event-driven integrations
-

12. Summary

Azure Functions:

- Enable **serverless, event-driven development**
- Reduce operational overhead
- Scale automatically
- Integrate deeply with Azure services
- Are well-suited for microservices and cloud-native architectures

